

TRACECHAIN: REAL-WORLD SUPPLY CHAIN CASE STUDY

End-to-End Persona Narrative, Smart Contract Endpoints & Flowchart Architectural Specs

System Overview: TraceChain is a Web3 supply chain tracking DApp built on the Polygon blockchain. It combines OpenZeppelin AccessControl, IPFS (via Pinata) for decentralized immutable metadata, and JSONBin for lightweight role requests to deliver cryptographically verifiable end-to-end product traceability.

1. Real-World Case Study: The Journey of Coffee Batch #402

Follow the real-life journey of **Specialty Colombian Arabica Coffee Batch #402** from high-altitude farms in Colombia to a specialty roaster in Seattle, WA.

Persona	Role & Address	Key Actions & Responsibility
Elena	DEFAULT_ADMIN_ROLE 0x71C8...8384	Oversees network security; approves verified role request applications on-chain.
Mateo	MANUFACTURER_ROLE 0x90F7...4b92	Harvests coffee at El Paraíso Estate; pins image & lab metrics to IPFS; creates Product #402.
Carlos	DISTRIBUTOR_ROLE 0x15d3...7A6D	Manages international freight transit from Port of Buenaventura to Seattle Freight Hub.
Sophia	RETAILER_ROLE 0x9965...205b	Store Manager at Emerald City Roasters; confirms final delivery & locks contract state.
Liam	End Consumer Public Auditor	Scans QR code on retail coffee bag to audit complete tamper-proof journey.

Act I: On-Chain Role Verification

Mateo runs El Paraíso Estate in Huila, Colombia. Before listing his specialty harvest, he connects his wallet to TraceChain and submits a role application requesting `MANUFACTURER_ROLE`. Elena, the network admin, inspects Mateo's fair-trade certifications and executes `grantRole(MANUFACTURER_ROLE, 0x90F7...)` on the Polygon smart contract. Mateo's wallet address is now permissioned on-chain.

Act II: Product Creation & Decentralized Pinning

Mateo harvests 500kg of organic micro-lot coffee beans. He takes high-resolution images of the moisture analysis report and bean batch. The TraceChain frontend pins the image file to IPFS via Pinata (CID: `QmX7b8...`) and constructs a standardized JSON metadata payload containing harvest altitude (1,850m), cup score (88.5), and estate coordinates. Upon calling `createProduct('ipfs://QmMeta402...')`, Smart Contract Product ID **#402** is generated with status **CREATED (0)**.

Act III: Logistics Transit & Custody Transfer

Carlos, a freight logistics manager at Pacific Global Shipping, applies for `DISTRIBUTOR_ROLE` and is approved by Elena. Mateo loads Container #C-902 onto a vessel at Port of Buenaventura. Mateo calls `transferOwnership(402, 0x15d3...)` to assign custody to Carlos. Carlos accepts custody and updates the status to **IN_TRANSIT (1)** with location string 'Port of Buenaventura -> Pacific Ocean Transit'.

Act IV: Store Receiving & Terminal Lock

Container #C-902 arrives at Seattle Freight Hub. Carlos transfers custody to Sophia (Store Manager at Emerald City Roasters, holding `RETAILER_ROLE`) and sets status to **PENDING_RETAILER (2)** with location 'Seattle Port Terminal 18'. Upon physical inspection, Sophia receives the container and calls `updateStatus(402, DELIVERED, 'Emerald City Roasters, Seattle WA')`. The smart contract enforces a terminal status lock — Product #402 can never be modified or reset again.

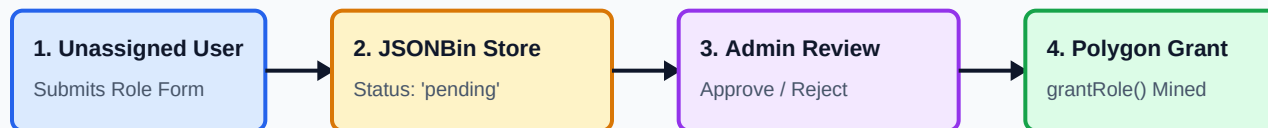
Act V: Consumer Verification

Liam buys a bag of coffee at Emerald City Roasters. He scans the printed QR code on the bag pointing to `/products/402`. The DApp retrieves the immutable history directly from Polygon Amoy testnet, showing every custody transfer timestamp, GPS location, and IPFS lab certificate with zero central server dependency.

2. Flowcharts & System Architecture Diagrams

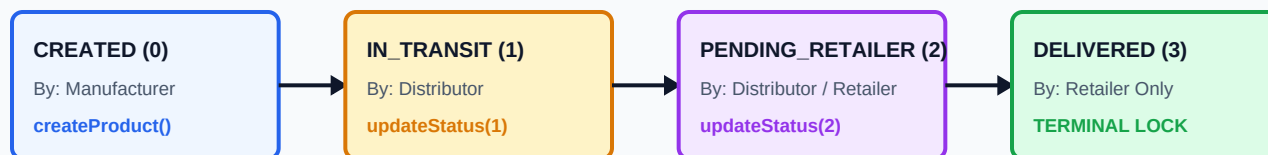
The diagrams below illustrate the two core workflows powering TraceChain: Role Authorization & Request Governance, and the Monotonic Product Lifecycle.

FLOWCHART 1: ROLE REQUEST & ON-CHAIN GOVERNANCE



API Endpoints: `POST /jsonbin/v3/b -> grantRole(bytes32, address) -> hasRole(role, account) = true`

FLOWCHART 2: MONOTONIC PRODUCT LIFECYCLE & CUSTODY TRANSFERS



Contract Rule: `status > currentStatus` required. **DELIVERED (3) & CANCELLED (4)** block all future edits.

3. Technical Flow Endpoints & API Specifications

Below is the complete technical interface specification for all smart contract methods, decentralised IPFS pinning calls, and role request REST endpoints.

Endpoint / Method Signature	Protocol / Contract	Inputs & Parameters	Access Control
<code>createProduct(string calldata _metadataCID)</code>	SupplyChain.sol	<code>_metadataCID</code> (IPFS hash string)	MANUFACTURER_ROLE
<code>updateStatus(uint256 _id, State _status, string _loc)</code>	SupplyChain.sol	<code>_id</code> (uint256), <code>_status</code> (enum 0-4), <code>_loc</code> (string)	DISTRIBUTOR / RETAILER
<code>transferOwnership(uint256 _id, address _newOwner)</code>	SupplyChain.sol	<code>_id</code> (uint256), <code>_newOwner</code> (address)	Current Custodian Only
<code>grantRole(bytes32 role, address account)</code>	SupplyChain.sol	<code>role</code> (bytes32 keccak), <code>account</code> (address)	DEFAULT_ADMIN_ROLE
<code>hasRole(bytes32 role, address account)</code>	SupplyChain.sol	<code>role</code> (bytes32), <code>account</code> (address)	Public View (Free)

POST /pinning/pinJSONtoIPFS	Pinata IPFS API	JSON Body: { name, description, image, attributes }	JWT Bearer Auth
GET /v3/b/{BIN_ID}/latest	JSONBin.io REST	Headers: X-Master-Key	Public / Admin API Key
PUT /v3/b/{BIN_ID}	JSONBin.io REST	JSON Body: Array of Role Request Objects	Master Key Auth

Standardized IPFS Metadata JSON Payload Schema:

```
{
  "name": "Specialty Colombian Arabica Coffee #402",
  "description": "Single-origin shade-grown coffee from El Paraíso Estate, Huila, Colombia.",
  "image": "ipfs://QmX7b8Yz9N3kLm2pQ4rS5tU6vW7x8y9z0a1b2c3d4e5f6",
  "attributes": [
    { "trait_type": "Origin", "value": "Huila, Colombia" },
    { "trait_type": "Altitude", "value": "1,850m" },
    { "trait_type": "Cup Score", "value": 88.5 },
    { "trait_type": "Batch Size", "value": "500 kg" }
  ]
}
```

4. Multi-User Role Testing Matrix & Verification

To rigorously test all role permissions and state transitions, use the following standardized 5-account test matrix on Polygon Amoy testnet.

Test Account	Role Assigned	Expected Dashboard View	Primary Allowed Operations
Account 1 0x71C8...8384	DEFAULT_ADMIN	AdminDashboard	Grant/Revoke Roles, Approve/Reject Requests, Audit All Products
Account 2 0x90F7...4b92	MANUFACTURER	ManufacturerDashboard	Register Product, Pin IPFS Metadata, Transfer Custody to Distributor
Account 3 0x15d3...7A6D	DISTRIBUTOR	DistributorDashboard	Update Status to IN_TRANSIT (1) / PENDING_RETAILER (2), Transfer Custody
Account 4 0x9965...205b	RETAILER	RetailerDashboard	Update Status to DELIVERED (3) -> Triggers Terminal Lock
Account 5 0x3C44...354E	NONE (Unassigned)	NoRoleDashboard	Submit Role Request Form, Browse Public Product Ledger

Polygon Amoy Gas Override Rules & Ethers v6 Execution Rules

Polygon Amoy EIP-1559 Enforcement: Polygon nodes enforce a minimum priority fee of 30 Gwei (30,000,000,000 wei). All write operations (grantRole, createProduct, updateStatus, transferOwnership) execute via getWriteContract() with gas overrides:

```
const overrides = await getGasOverrides(); // maxPriorityFeePerGas: 30 Gwei, maxFeePerGas: 60 Gwei
const tx = await writeContract.updateStatus(productId, status, location, overrides);
```

Document Verification & Repository Specs:

- GitHub Repository: <https://github.com/pranil-shripad/TraceChain>
- Polygon Amoy Deployed Contract: 0x9cCd86A9117621c8B3D063A41ef8DEb3c43F9Bf9
- Live Docker Container Service: <http://localhost:8080>